03/26/2026: Diffusion Overview

Reference: *Deep Learning (Sec 20.1-20.2)*

CSCI 546: Diffusion Models

Table of contents

1. Course Logistics
2. Group Work Part I
3. Lightning Chat
4. Mini-Lecture
5. Group Work Part II
6. Coding Up Diffusion

Course Logistics

Announcements

- I have previous reading surveys to return.

Announcements

- I have previous reading surveys to return.
- One reading survey will be dropped from your total score when computing your course grade. This can help to cover absences for any reason, or just an off day.

Announcements

- I have previous reading surveys to return.
- One reading survey will be dropped from your total score when computing your course grade. This can help to cover absences for any reason, or just an off day.
- Please get back to me by **4pm today** on if/when you want to retake the midterm.

Announcements

- I have previous reading surveys to return.
- One reading survey will be dropped from your total score when computing your course grade. This can help to cover absences for any reason, or just an off day.
- Please get back to me by **4pm today** on if/when you want to retake the midterm.
- If relevant, please check in with your lightning chat co-presenter to determine if you want to present together (preferred) or separately.

Reading Survey

1. Describe the forward encoder of a diffusion model. (Sec 20.1)
2. Describe the reverse decoder of a diffusion model. (Sec 20.2)

Group Work Part I

Random groups

Aubrey Williams: 8
Austin Barton : 5
Blake Sigmundstad: 6
Diego Moylan: 7
Dillon Shaffer: 7
Ismoiljon Muzaffarov: 2
Jacob Tanner: 1
Josh Stoneback: 8
Joshua Bowen: 6
Joshua Culwell: 4
Laura Banaszewski: 4
Lina Hommel: 5
Logan Racz: 1
Matt Hall: 2
Mike Kadoshnikov: 2
Owen Cool: 3
Racquel Bowen: 1
Samuel Mocabee: 3
Tatiana Kirillova: 3

Group Exercises (Part I)

1. How do we know that if we define the forward process via the transition kernel $q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t; \sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t \boldsymbol{I})$ that there is an equilibrium distribution, and moreover that the equilibrium distribution is $\pi = \mathcal{N}(0, \boldsymbol{I})$?

Group Exercises (Part I)

1. How do we know that if we define the forward process via the transition kernel $q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t; \sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t \boldsymbol{I})$ that there is an equilibrium distribution, and moreover that the equilibrium distribution is $\pi = \mathcal{N}(0, \boldsymbol{I})$?

Hint: (Assuming that the kernel is aperiodic and π-irreducible), it suffices to verify detailed balance:

That is, we must show

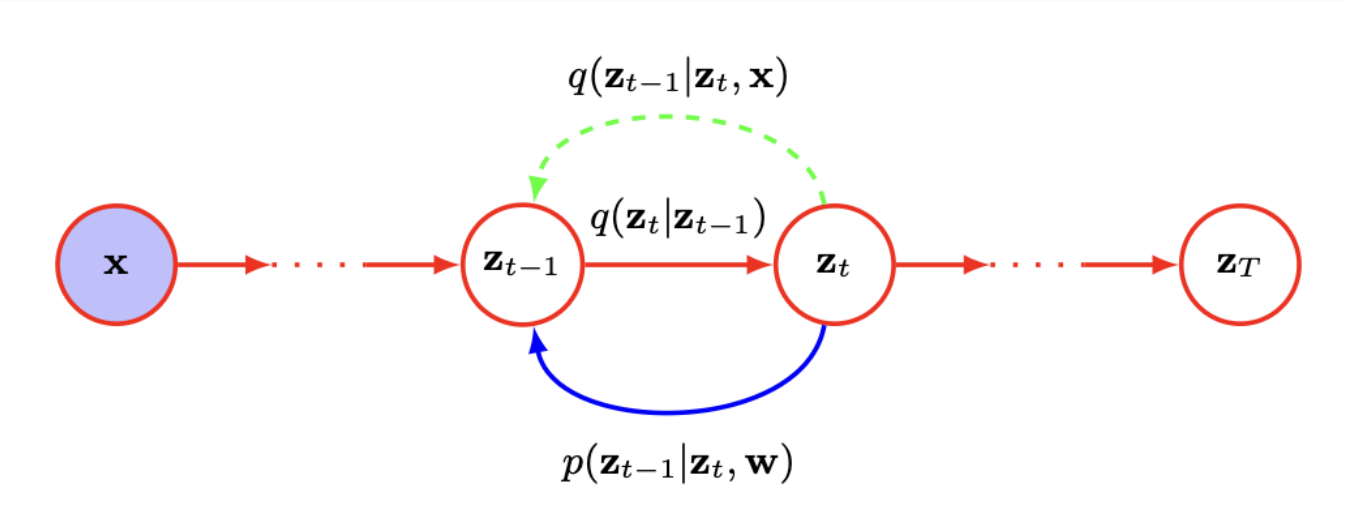
$$\pi(y)q(x \mid y) \stackrel{?}{=} \pi(x)q(y \mid x)$$

Lightning Chat

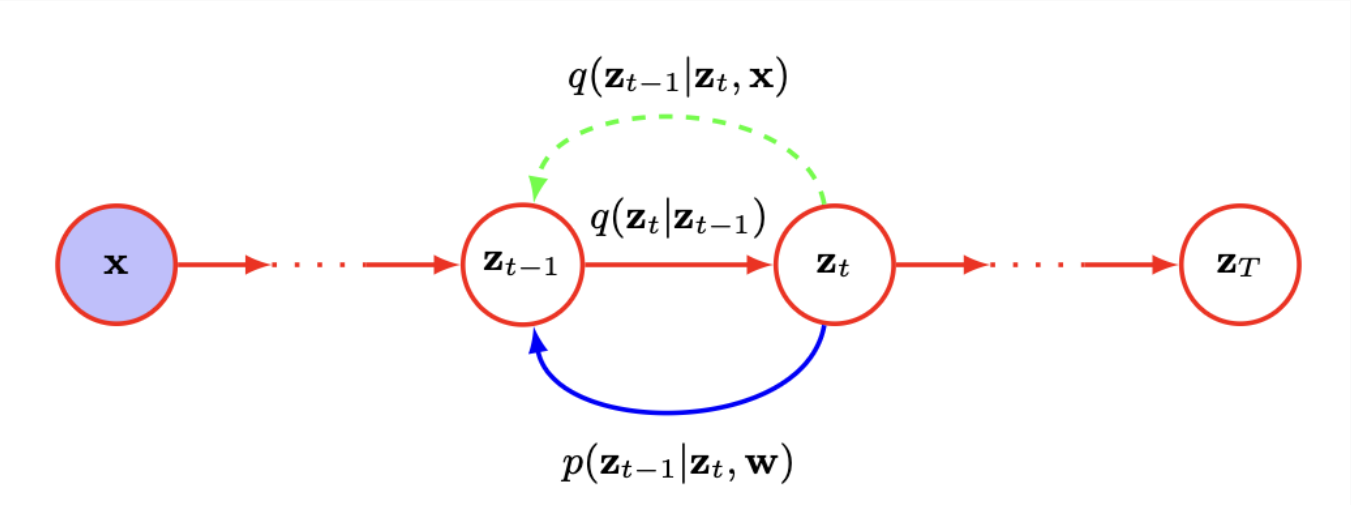
Lightning Chat

Mini-Lecture

Forward Encoder



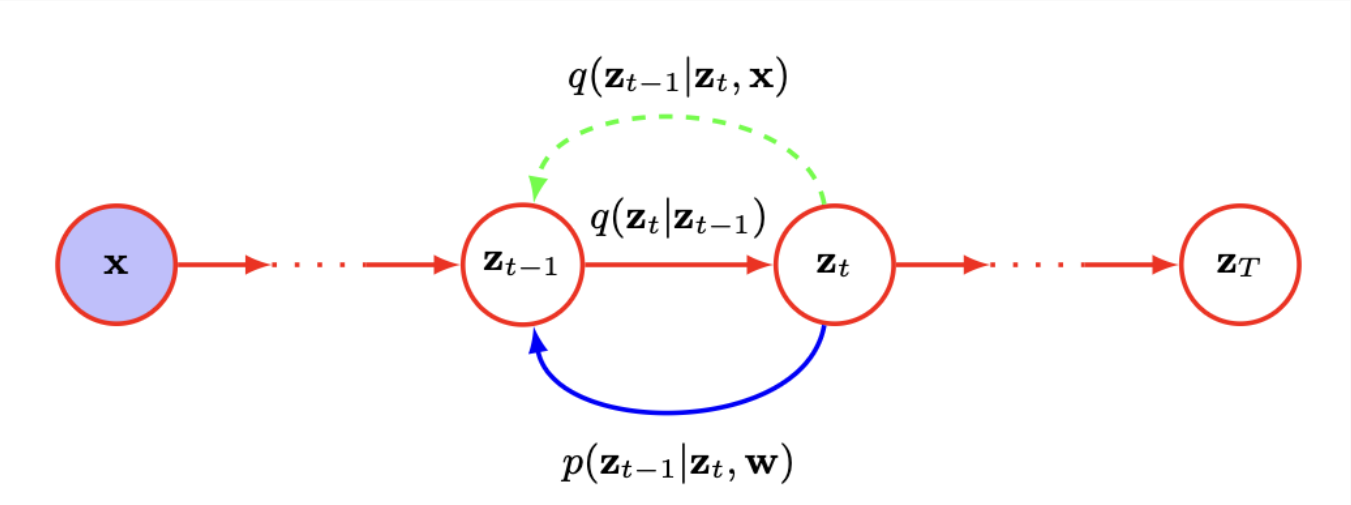
Forward Encoder



The **forward transition kernel** is given by

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\boldsymbol{z}_{t-1}, \beta_t \boldsymbol{I}\right) \tag{1}$$

Forward Encoder

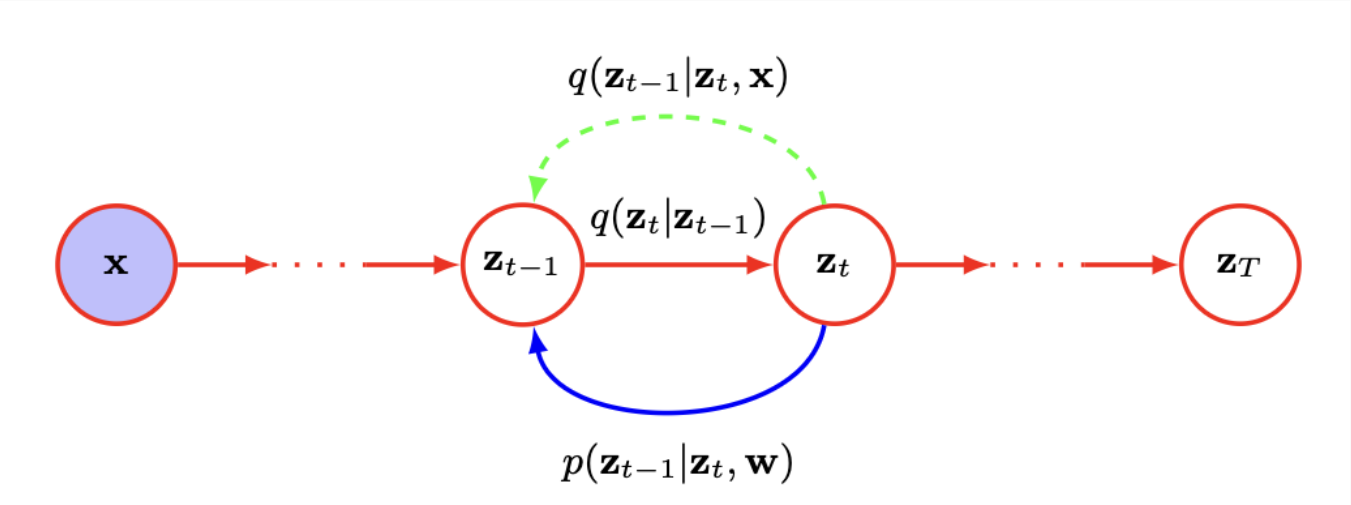


The **forward transition kernel** is given by

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\,\boldsymbol{z}_{t-1}, \beta_t \boldsymbol{I}\right) \tag{1}$$

(for $t = 1, \ldots, T$ if we define $\boldsymbol{z}_0 := \boldsymbol{x}$)

Forward Encoder



The **forward transition kernel** is given by

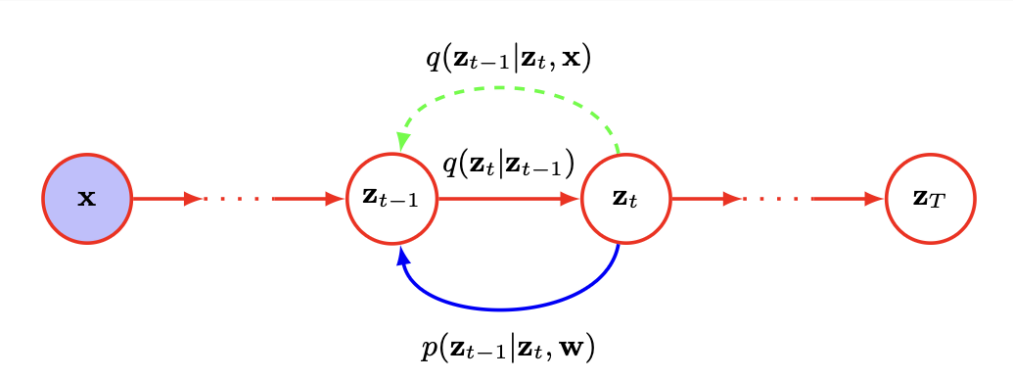
$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\boldsymbol{z}_{t-1}, \beta_t \boldsymbol{I}\right) \tag{1}$$

As we will show (group exercise), by marginalizing over intermediate variables $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$, we obtain the **diffusion kernel** as:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right) \tag{2}$$

$$\text{where} \quad \alpha_t := \prod_{\tau=1}^{t} (1-\beta_t)$$

Forward Encoder



The **forward transition kernel** is given by

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\boldsymbol{z}_{t-1}, \beta_t \boldsymbol{I}\right) \tag{1}$$

As we will show (group exercise), by marginalizing over intermediate variables $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$, we obtain the **diffusion kernel** as:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right) \tag{2}$$

$$\text{where} \quad \alpha_t := \prod_{\tau=1}^{t} (1-\beta_t)$$

Poll. What happens to Eq. (4) as t increases?

Forward Encoder

The **forward transition kernel** is given by

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\boldsymbol{z}_{t-1}, \beta_t \boldsymbol{I}\right) \tag{1}$$

As we will show (group exercise), by marginalizing over intermediate variables $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$, we obtain the **diffusion kernel** as:

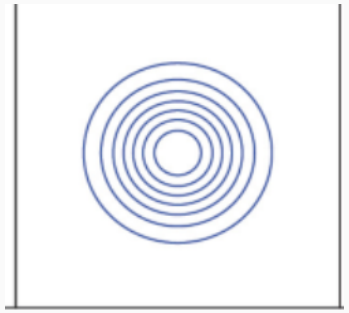
$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right) \tag{2}$$

$$\text{where} \quad \alpha_t := \prod_{\tau=1}^{t} (1-\beta_t)$$

Poll. What happens to Eq. (4) as t increases?

Solution.

- the mean shrinks towards 0
- the covariance goes to identity matrix.



Forward Encoder

The **diffusion kernel** is obtained by marginalizing over $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right), \qquad \text{where} \quad \alpha_t := \prod_{\tau=1}^{t}(1-\beta_t)$$

Forward Encoder

The **diffusion kernel** is obtained by marginalizing over $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right), \qquad \text{where} \quad \alpha_t := \prod_{\tau=1}^{t}(1-\beta_t)$$

Poll. Why do we care?

Forward Encoder

The **diffusion kernel** is obtained by marginalizing over $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right), \qquad \text{where} \quad \alpha_t := \prod_{\tau=1}^{t}(1-\beta_t)$$

Poll. Why do we care?

Solution.

We see that each intermediate distribution has a simple closed-form Gaussian expression from which we can directly sample, which will prove useful when training DDPMs as it allows efficient stochastic gradient descent [...] without having to run the whole chain.

$$\mathcal{L}(\mathbf{w}) = \underbrace{\int q(\mathbf{z}_1|\mathbf{x}) \ln p(\mathbf{x}|\mathbf{z}_1, \mathbf{w}) \, \mathrm{d}\mathbf{z}_1}_{\text{reconstruction term}} - \underbrace{\sum_{t=2}^{T} \int \mathrm{KL}(q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}) \| p(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{w})) q(\mathbf{z}_t|\mathbf{x}) \, \mathrm{d}\mathbf{z}_t}_{\text{consistency terms}} \tag{20.32}$$

Group Work Part II

Group Exercises (Part II)

Recall. The **forward transition kernel** is given by

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\boldsymbol{z}_{t-1}, \beta_t \boldsymbol{I}\right) \tag{3}$$

The **diffusion kernel** is obtained by marginalizing over $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{t-1}$:

$$q(\boldsymbol{z}_t \mid \boldsymbol{x}) = \mathcal{N}\left(\sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right), \qquad \text{where} \quad \alpha_t := \prod_{\tau=1}^{t}(1-\beta_t) \tag{4}$$

Exercise. Show that the marginal distribution of the forward process Eq. (3) is given by Eq. (4).

- *Hint:* Use induction.
- *Hint:* Use the fact that the sum of two independent Gaussian random variables is itself a Gaussian with additive means and covariances.
- *Hint:* To use the above, rewrite Eq. (3) as

$$\boldsymbol{z}_t = \sqrt{1-\beta_t}\boldsymbol{z}_{t-1} + \beta_t \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \boldsymbol{I})$$

Coding Up Diffusion

Coding Up Diffusion

Goal. The goal between now and next class: code up a simple diffusion model!

Coding Up Diffusion

Goal. The goal between now and next class: code up a simple diffusion model!

1. *Path 1*: Play with Emilio's code. Some things you might do:
   - Get it to work.
   - Try a different dataset (e. g. swiss roll, MNIST images)
   - Tweak it to run binomial diffusion, apply to binary heartbeat data, as per Sohl-Dickstein et al (2015) Fig 2.
   - Adjust codebase per ideas from Bishop.

Coding Up Diffusion

Goal. The goal between now and next class: code up a simple diffusion model!

1. *Path 1*: Play with Emilio's code. Some things you might do:
   - Get it to work.
   - Try a different dataset (e. g. swiss roll, MNIST images)
   - Tweak it to run binomial diffusion, apply to binary heartbeat data, as per Sohl-Dickstein et al (2015) Fig 2.
   - Adjust codebase per ideas from Bishop.
2. *Path 2*: Code up Bishop's algorithms from scratch, using Emilio's code as a guide, and apply it to some simple dataset.

Coding Up Diffusion

Goal. The goal between now and next class: code up a simple diffusion model!

1. *Path 1*: Play with Emilio's code. Some things you might do:
   - Get it to work.
   - Try a different dataset (e. g. swiss roll, MNIST images)
   - Tweak it to run binomial diffusion, apply to binary heartbeat data, as per Sohl-Dickstein et al (2015) Fig 2.
   - Adjust codebase per ideas from Bishop.
2. *Path 2*: Code up Bishop's algorithms from scratch, using Emilio's code as a guide, and apply it to some simple dataset. Note: this is harder because Bishop's algorithms are based on "predicting the noise" (Sec 20.2.4).

Coding Up Diffusion

Goal. The goal between now and next class: code up a simple diffusion model!

1. *Path 1*: Play with Emilio's code. Some things you might do:
   - Get it to work.
   - Try a different dataset (e. g. swiss roll, MNIST images)
   - Tweak it to run binomial diffusion, apply to binary heartbeat data, as per Sohl-Dickstein et al (2015) Fig 2.
   - Adjust codebase per ideas from Bishop.
2. *Path 2*: Code up Bishop's algorithms from scratch, using Emilio's code as a guide, and apply it to some simple dataset. Note: this is harder because Bishop's algorithms are based on "predicting the noise" (Sec 20.2.4).

Format. Work in groups of two. Tell me your partner before leaving. Prepare a **5 min** (hard cap!) presentation for next class showing us what you tried/learned. Note: This will probably just be 2-3 slides.